



Carnegie Mellon University
School of Computer Science
Executive Education

Module 7: Capstone Project

Self-Study Capstone Checkpoint Activity 7.1: Final Capstone Planning

Final capstone report planning

1. **Project title**

Write a clear and descriptive title for your capstone project.

DragonWings Power Availability Profile (PAP) Forecaster

An autonomous, read-only agent system for explaining available solar power.

Shawn Cunningham

Agentic AI Program - Final Capstone, Section B

Updated September 8, 2026

Implementation baseline: 1a7a7ee

Completed planning template, version 2

2. **Problem and user**

State the real-world problem your autonomous system addresses. Identify the intended user or users, and explain why this problem matters.

DragonWings operators need to understand how much additional electrical demand a portable solar and battery system might support. A dashboard showing current generation, load and battery voltage cannot by itself answer a twelve-hour planning question or explain why an estimate should be trusted.

PAP turns those readings into a time-based estimate, supporting evidence and a clear reason when it withholds a result. The intended users are site operators planning optional loads and engineers reviewing system behavior. The aim is to make power availability understandable while keeping equipment decisions with the operator. PAP has no equipment-control path.

3. **System goal and scope**

Explain what your system is designed to do, what successful performance looks like, and any important boundaries or constraints you defined.

The goal is a twelve-hour profile of additional solar power in kW and interval energy in kWh. Once started, the workflow acquires evidence, validates inputs, calculates availability, invokes bounded interpretation when needed, and publishes or withholds.

The MVP supports five wings, DW 1.21-1.25, using their latest persisted MySolArk scrape or historical replay. Operators select start/end dates within August 30-September 6, 2026 (Pacific, end excluded), one-hour or fifteen-minute steps, and playback speed. Start/Pause/Resume advances one selected wing automatically.

Success means correct arithmetic and units, visible source provenance, rejection of stale or incomplete evidence, enforcement of voltage floors, bounded model work, and inspectable saved runs. Demand stays at its measured value; battery discharge is not allocated. The system provides decision support, not a guarantee that equipment can accept an extra load.

4. **Final system architecture**

Describe the final design of your system and how its major components work together. Depending on your project, this may include reasoning loops, memory, tools, retrieval, Tree-of-Thought reasoning, multi-agent coordination, guardrails, logging, evaluation, and human intervention.

LangGraph connects read-only MCP evidence, deterministic forecasting and constraint checks, optional model guidance, and publication. PostgreSQL stores evidence/results; its checkpointing supports resume. T8 evaluates later measured power; T9 uses pgvector to retrieve qualified memory.

LLM scenarios:

- Used: error thresholds trigger generator/critic advice.
- Used: optional interpretation is enabled on a valid calculation.
- Skipped: normal run, no escalation, interpretation disabled.
- Skipped: stale/missing data or voltage-floor failure; withhold.
- No LLM: arithmetic, unit conversion and T3/T5/T6 authority.

LangChain agents provide bounded interpretation and selective Tree-of-Thought guidance; they cannot change calculated power or command equipment. Inspector links Context, Memory, Tools, Subagent and Trace to saved runs; Health checks current dependencies. LangSmith is optional.

5. Design evolution across the program

Review your earlier checkpoints and explain how your system evolved from Module 1 through Module 6. Highlight the most important refinements, additions, or changes, and explain why they improved the system.

Checkpoint 1 framed the operator's power-planning problem. Checkpoint 2 developed an evidence-and-feedback cycle with tools and persistent memory. Checkpoint 3 added retrieval of qualified guidance and outcomes. Checkpoint 4 reserved bounded alternative search for ambiguity. Checkpoint 5 separated generator and critic roles; the MVP adds optional interpretation for comparison. Checkpoint 6 layered validation, evaluation and human oversight.

Feedback on Checkpoint 6.1 led to calibration maintenance: recent error windows, a drift proxy, review status and a read-only threshold preview. Operators still retune thresholds explicitly.

Integration refined the MVP with five-wing replay, Pacific-time graphs, separate Forecast and Inspector pages, and stored Open-Meteo weather. Synthetic weather remains only in the sunny demonstration and older saved episodes. Voltage replaces SOC as the battery-state input.

6. Implementation overview

Summarize how you built the system. Include the frameworks, tools, models, application programming interfaces (APIs), or libraries you used, and explain how they support your design.

The stack uses Python/FastAPI, LangGraph, LangChain, PostgreSQL/pgvector and a native HTML/CSS/JavaScript UI. It runs from the command line without Docker or Gradio. AGENT_BACKEND selects ollama, openai or anthropic; AGENT_MODEL selects the model. Local demos used Gemma 3 4B. ENABLE_INTERPRETATION_AGENT toggles the third role (default off).

PAP reads stored MySolArk telemetry and Open-Meteo-derived weather through read-only database connections with three-second connection/query limits. PAP makes no weather HTTP request. Replay selects observations at/before each hourly start, within 90 minutes. Live prefers twelve complete forecast hours updated within 24 hours. Incomplete weather withholds; there is no silent synthetic fallback.

Solar factor = $\min(1, \text{interval input} / \text{first input})$, using irradiance for observations or capacity factor for forecasts. A zero reference gives zero factors. Measured solar power is multiplied by the factor. New weather and forecast-version identities preserve old episodes and isolate their outcome feedback.

7. Evaluation and results

Explain how you evaluated your system. Describe the criteria, tests, or examples you used to assess performance, reliability, and limitations. Summarize the main results.

At baseline 1a7a7ee, make verify passed 78 backend tests and 22 Playwright tests. Deterministic doubles avoid live provider/source dependencies. Checks cover arithmetic, weather, withholding, provenance, history, calibration and UI workflows. [1]

Five August 30 noon-Pacific source spot checks, one per wing, passed T3 with twelve stored-observation intervals each; four night intervals had zero factors. A live DW 1.24 check returned twelve stored forecast hours. These confirm source integration, not accuracy. [2]

One earlier local-model comparison used synthetic weather. Two roles: four calls, 53.035 seconds, published. Three roles: six calls, 70.223 seconds, withheld. Calculated power stayed identical; this single sequential trial establishes no quality benefit. [3]

Replay uses actual weather across the horizon: retrospective simulation, not an as-of forecast benchmark. No new full-week accuracy or controlled cloud comparison is claimed.

8. Safety and reliability considerations

Explain the guardrails, monitoring, fallback logic, and human oversight you included to improve safety and dependability.

Deterministic T3/T5/T6 checks remain authoritative over model confidence. Stale telemetry, incomplete weather or a battery voltage at/below the selected wing's fixed floor withholds additional power. Floors come from positive observed minima and never decrease automatically; they are an operating policy, not a calibrated SOC conversion. Equipment caps apply when configured, and battery-discharge allowance is zero.

Source access is read-only. PAP responses exclude source credentials, device identifiers and raw source JSON. Models receive structured evidence with no hardware-control or unrestricted SQL tools. Search has at most eight shared model attempts, bounded depth and per-call time limits. Inspector records decisions and evidence rather than private chain-of-thought.

Calibration compares recent and prior error windows, expires old feedback and flags review needs. Operators preview and explicitly change thresholds; tuning is not automatic. ECE is unavailable because confidence labels are qualitative. Human review remains necessary for explanations, threshold maintenance and any equipment decision.

9. **Limitations and next steps**

Identify the current limitations of your system and suggest realistic next steps for improving it.

Capped persistence cannot amplify morning generation or predict sunrise from zero solar power. Demand is constant, future battery voltage is unmodeled, and live equipment caps are unconfigured. Scrape timestamps are not verified device measurement times. Missing weather can increase withholding.

Replay uses future actual weather; older synthetic-weather results remain historical evidence. Error drift is a proxy, not a formal distribution-shift test.

Next: an irradiance/array-based forecast, verified equipment limits, held-out as-of forecasts across seasons, repeated order-balanced provider/agent comparisons, and operator usability review. Confidence calibration first requires meaningful probabilities and labeled outcomes.

10. **Public GitHub repository**

Prepare a public GitHub repository for your capstone project, and include the link in your final report. Your repository should make it possible for a technical audience to understand your project and review your implementation. At a minimum, it should include:

1. A README that explains the project, problem, architecture, setup steps, and usage
2. The main code or notebooks for your system
3. Any sample inputs, outputs, or evaluation artifacts you want to share
4. Clear instructions for running or reviewing the project

<https://github.com/roboyak/PAP-Agent>

Implementation: 1a7a7ee (September 8, 2026).

README.md and howto.md explain architecture, setup and UI use. Core code is in src/pap_agent; tests contains backend/browser checks; docs holds design/evaluation records; submission holds artifacts and builders.

Evidence: [1] docs/pr/PR-25.md

[2] docs/data/stored-weather-smoke.json

[3] docs/pr/PR-11.md

Native setup: make bootstrap. Verification: make verify.

Sunny demo: PAP_PROFILE=macmini-replay make run.

Real data requires the authorized local source database.

The repository is private during preparation. The owner will make it public and verify signed-out access before submission.